

Object Oriented Programming

1 Class & Object

- **Class** → Blueprint
- **Object** → Instance of class

Example: Car blueprint = class, actual car = object 🚗

2 Constructor (`__init__`)

- Automatically called when object created
- Used to initialize variables

3 Encapsulation

- Data hiding using **private variables** (`__var`)
- Access using methods

4 Inheritance

- Child class inherits parent properties
- Code reuse

5 Polymorphism

- Same function name, different behavior
- Example: `sound()` → Dog & Cat

6 Abstraction

- Hide implementation
- Show only required functionality
- Use `ABC` module

Class

A Python class is a group of attributes and methods(function).

What is Attribute ?

Attributes are represented by variable that contains data.

What is Method?

Method performs an action or task. It is similar to function.

What is a Function?

A function in Python is a block of reusable code that performs a specific task.

```
def add (x,y):
```

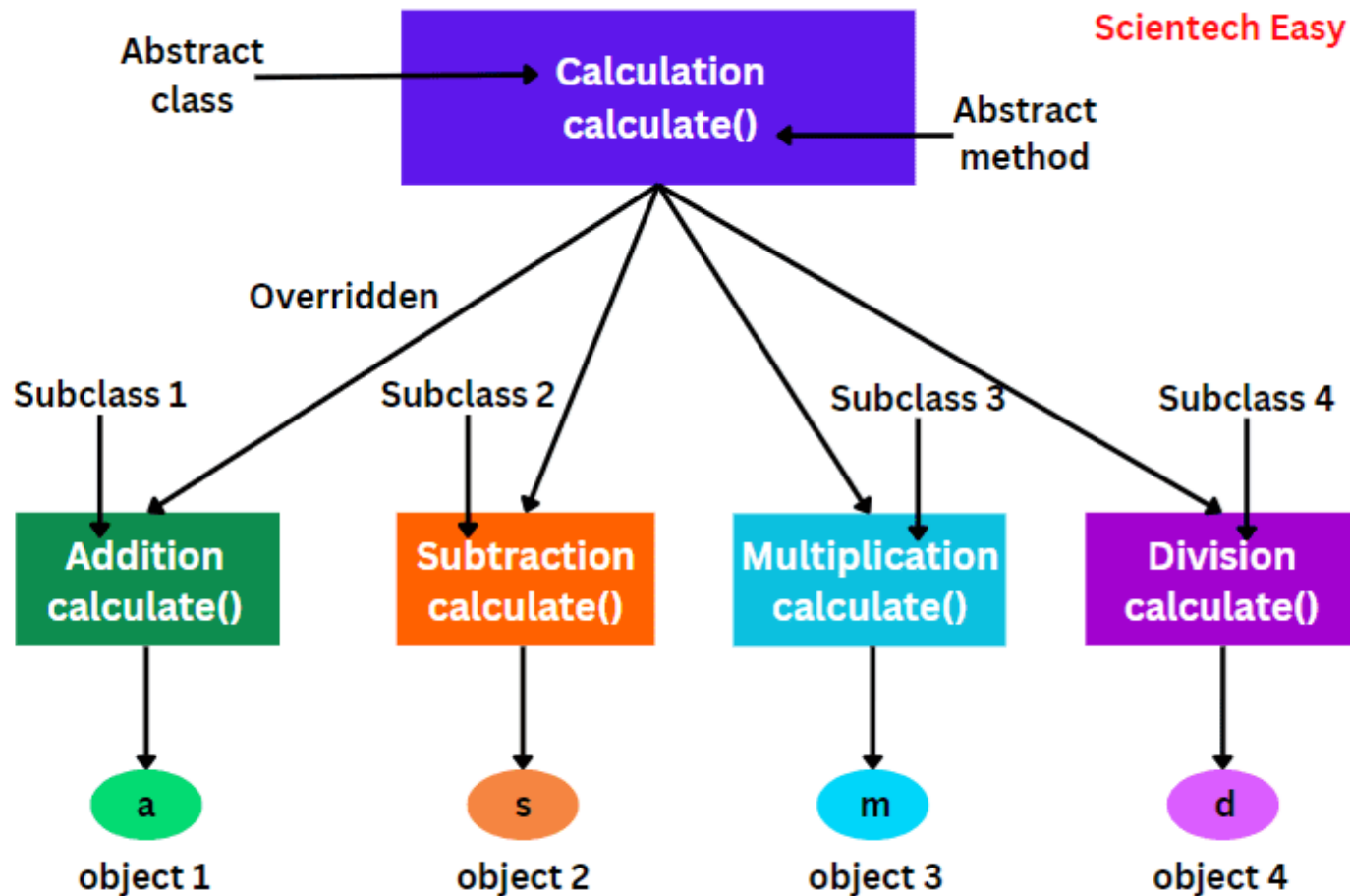
```
    return(x+y)
```

```
print(add(12,15))
```

```
print(add(15,105))
```

```
print(add(200,300))
```

```
a = add(500,300)
```



Base class

How to Create Class

```
class Classname(object) :
```

```
def __init__(self):
```

constructor

```
    self.variable_name = value
```

```
    self.variable_name = 'value'
```

Attributes

Method

```
def method_name(self):
```

```
    Body of Method
```

```
class Classname :
```

```
def __init__(self):
```

```
    self.variable_name = value
```

```
    self.variable_name = 'value'
```

```
def method_name(self):
```

```
    Body of Method
```

- class - class keyword is used to create a class
- object - object represents the base class name from where all classes in Python are derived. This class is also derived from object class. This is optional.
- __init__() – This method is used to initialize the variables. This is a special method. We do not call this method explicitly.
- self – self is a variable which refers to current class instance/object.

How to Create Class

```
class Mobile:
    # Constructor
    def __init__(self, m):
        self.model = m
        self.model_1 = 'Realme X'

    def show_model(self, p):
        price = p
        print("Model:", self.model, "and Price:", price)
```

constructor (points to `def __init__(self, m):`)

Instance Variable (points to `self.model = m`)

Formal Argument (points to `p` in `def show_model(self, p):`)

Instance Method (points to `def show_model(self, p):`)

Accessing Instance Variable (points to `self.model` in `print("Model:", self.model, "and Price:", price)`)

Local Variable (points to `price = p`)

object (points to `realme = Mobile('Realme X')`)

~~# Accessing Method from outside Class~~
`realme.show_model(1000)`

`# Passing Argument to Constructor`
`realme = Mobile('Realme X')`

Rules

- The class name can be any valid identifier.
- It can't be Python reserved word.
- A valid class name starts with a letter, followed by any number of letter, numbers or underscores.
- A class name generally starts with Capital Letter
- .

Object

- Object is class type variable or class instance. To use a class, we should create an object to the class.
- Instance creation represents allotting memory necessary to store the actual data of the variables.
- Each time you create an object of a class a copy of each variables defined in the class is created.
- In other words you can say that each object of a class has its own copy of data members defined in the class.

Syntax: -

```
object_name = class_name()
```

```
object_name = class_name(arg)
```


self Variable

self is a default variable that contains the memory address of the current object.

This variable is used to refer all the instance variable and method.

When we create object of a class, the object name contains the memory location of the object.

This memory location is internally passed to *self*, as *self* knows the memory address of the object so we can access variable and method of object.

self is the first argument to any object method because the first argument is always the object reference. This is automatic, whether you call it *self* or not.

```
def __init__(self):
```

```
def show_model(self):
```

Constructor

Python supports a special type of method called constructor for initializing the instance variable of a class.

A class constructor, if defined is called whenever a program creates an object of that class.

A constructor is called only once at the time of creating an instance.

If two instances are created for a class, the constructor will be called once for each instance.

Type of Variable

- Instance Variable
- Class Variable / Static Variable

Instance Variable

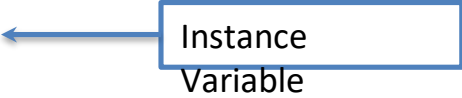
Instance variables are the variables whose separate copy is created in every object.
Instance variables are defined and initialized using a constructor with self parameter.

Ex:-

```
class Mobile:
```

```
    def __init__(self):
```

```
        self.model = 'RealMe X'
```



Instance
Variable

```
    def show_model(self):
```

```
        print(self.model)
```

```
realme = Mobile( )
```

Accessing Instance Variable

With Instance Method

To access instance variable, we need instance methods with self as first parameter then we can access instance variable using self.variable_name

class Mobile:

```
    def __init__(self):
```

```
        self.model = 'RealMe X'
```

Instance Variable

```
    def show_model(self):
```

Instance Method

```
        self.model
```

```
realme = Mobile()
```

Accessing Instance Variable

Accessing Instance Variable

Outside Class

We can access instance variable using object_name.variable_name

class Mobile:

```
def __init__(self):
```

```
    self.model = 'RealMe X'
```

Instance Variable

```
def show_model(self):
```

Instance Method

```
    self.model
```

Accessing Instance Variable

```
realme = Mobile()
```

```
realme.model
```

Accessing Instance Variable from outside Class

Class Variable / Static Variable

Class variables are the variables whose single copy is available to all the instance of the class. If we modify the copy of class variable in an instance, it will effect all the copies in the other instance.

Ex:-

```
class Mobile:
```

```
    fp = 'Yes'
```



Class Variable

```
    def __init__(self):
```

```
        self.model = 'RealMe X'
```

```
    def show_model(self):
```

```
        print(self.model)
```

```
realme = Mobile( )
```

Accessing Class/Static Variable

With Class Method

To access class variable, we need class methods with cls as first parameter then we can access class variable using cls.variable_name

class Mobile:

fp = 'Yes'

← Class Variable

def __init__(self):

self.model = 'RealMe X'

def show_model(self):

print(self.model)

@classmethod

← Class Method

def is_fp(cls):

cls.fp

← Accessing Class Variable inside Class Method

realme = Mobile()

Accessing Class/Static Variable

Outside Class

We can access class variable using Classname.variable_name

class Mobile:

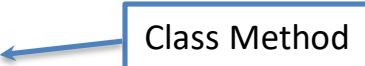
fp = 'Yes'

Class Variable

A blue arrow points from the text 'Class Variable' in a box to the variable 'fp' in the code 'fp = 'Yes''.

@classmethod

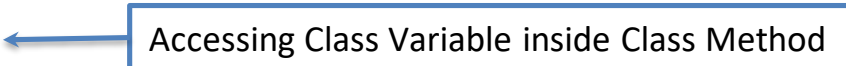
Class Method

A blue arrow points from the text 'Class Method' in a box to the '@classmethod' decorator in the code.

def show(cls):

cls.fp

Accessing Class Variable inside Class Method

A blue arrow points from the text 'Accessing Class Variable inside Class Method' in a box to the 'cls.fp' attribute access inside the 'show' method.

realme = Mobile()

Mobile.fp

Accessing Class Variable outside class

A blue arrow points from the text 'Accessing Class Variable outside class' in a box to the 'Mobile.fp' attribute access in the code.

Type of Methods

- Instance Methods
- Class Methods
- Static Methods

Instance Method

Instance methods are the methods which act upon the instance variables of the class. Instance method need to know the memory address of the instance which is provided through *self* variable by default as first parameter for the instance method.

Syntax:-

```
def method_name(self):  
    function body
```



Instance Method without Parameter/Formal Arguments

```
def method_name(self, f1, f2):  
    function body
```



Instance Method with Parameter/Formal Arguments

Instance Method with Parameter

```
class Mobile:
```

```
    def __init__(self):
```

```
        self.model = 'RealMe X'
```

```
    def show_model(self, p):
```

```
        self.price = p
```

```
        print(self.model, self.price)
```

```
realme = Mobile( )
```

Instance variable

Instance Method with parameter

Instance Variable

Parameter

Calling Instance Method with Argument

Syntax:- object_name.method_name(Actual_argument)

Ex:- realme.show_model(1000)

```
class Mobile:
```

```
    def __init__(self):
```

```
        self.model = 'RealMe X'
```

```
    def show_model(self, p):
```

```
        self.price = p
```

```
        print(self.model, self.price)
```

```
realme = Mobile( )
```

```
realme.show_model(1000)
```



Calling Method with argument

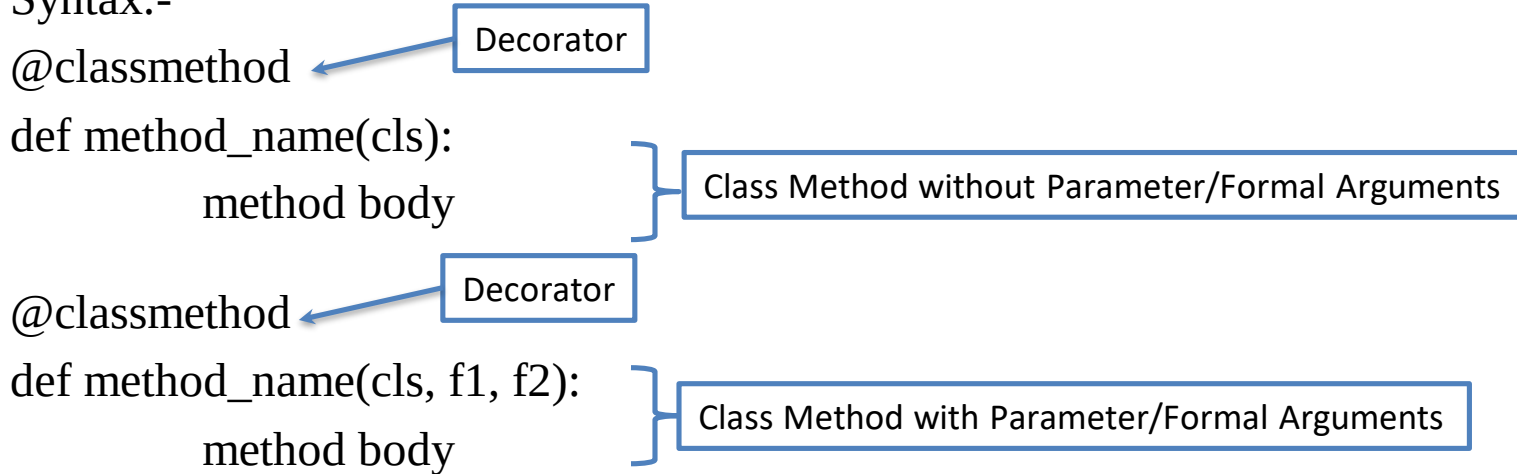
Class Methods

Class methods are the methods which act upon the class variables or static variable of the class.

Decorator `@classmethod` need to write above the class method.

By default, the first parameter of class method is `cls` which refers to the class itself.

Syntax:-



Calling Class Method with Argument

Syntax:- `Classname.method_name(Actual_argument)`

Ex:- `Mobile.show_model('4GB')`

```
class Mobile:
```

```
    fp = 'Yes'
```

```
    @classmethod
```

```
    def show_model(cls, r):
```

```
        cls.ram = r
```

```
        print(cls.fp, cls.ram)
```

```
realme = Mobile( )
```

```
Mobile.show_model(101)
```



Calling Method with argument

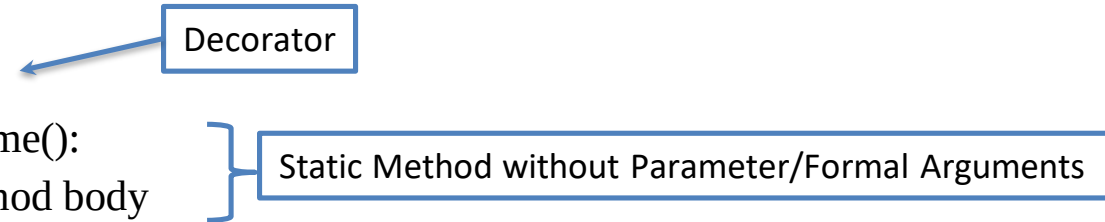
Static Methods

Static Methods are used when some processing is related to the class but does not need the class or its instances to perform any work.

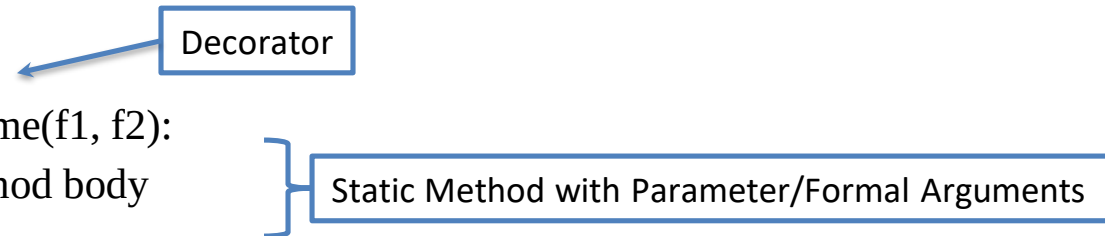
We use static method when we want to pass some values from outside and perform some action in the method.

Decorator `@staticmethod` need to write above the static method.

Syntax:-

`@staticmethod` 

```
def method_name():  
    method body
```

`@staticmethod` 

```
def method_name(f1, f2):  
    method body
```


Static Method without Parameter

```
class Mobile:
```

```
    @staticmethod
```

Decorator

```
    def show_model():  
        print("RealMe X")
```

Static Method

```
realme = Mobile( )
```

```
class Mobile:
```

```
    fp = 'Yes'
```

```
    @staticmethod
```

Static Method

```
    def show_model():  
        print(Mobile.fp)
```

```
realme = Mobile( )
```

Calling Static Method with Argument

Syntax:- Classname.method_name(Actual_argument)

Ex:- Mobile.show_model(1000)

```
class Mobile:
```

```
    @staticmethod
```

```
    def show_model(m, p):
```

```
        model = m
```

```
        price = p
```

```
        print(model, price)
```

```
realme = Mobile( )
```

```
Mobile.show_model('RealMe X', 1000)
```



Calling Method with argument

Nested Class

A class within a class is called as nested class or nesting of a class.

```
class OuterClassName:
```

```
    def __init__(self):
```

```
        self.variable_name = value
```

```
        self.innerClassObjectName = self.InnerClassName( )
```



Inner Class Object

```
    def method_name(self):
```

```
        method body
```

```
class InnerClassName:
```

```
    def __init__(self):
```

```
        self.variable_name = value
```

```
    def method_name(self):
```

```
        method body
```

```
class Army:
```

Outer Class

```
    def __init__(self):
```

```
        self.name = 'Rahul'
```

```
        self.gn = self.Gun()
```

Inner Class Object

```
    def show(self):
```

```
        print(self.name)
```

```
class Gun:
```

Inner Class

```
    def __init__(self):
```

```
        self.name = 'AK47'
```

```
        self.capacity = '75 Rounds'
```

```
        self.length = '34.3 in'
```

```
    def disp(self):
```

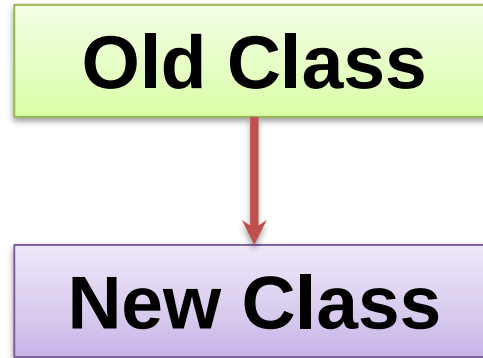
```
        print(self.name, self.capacity, self.length)
```

```
a = Army()
```

Outer Class Object

Inheritance

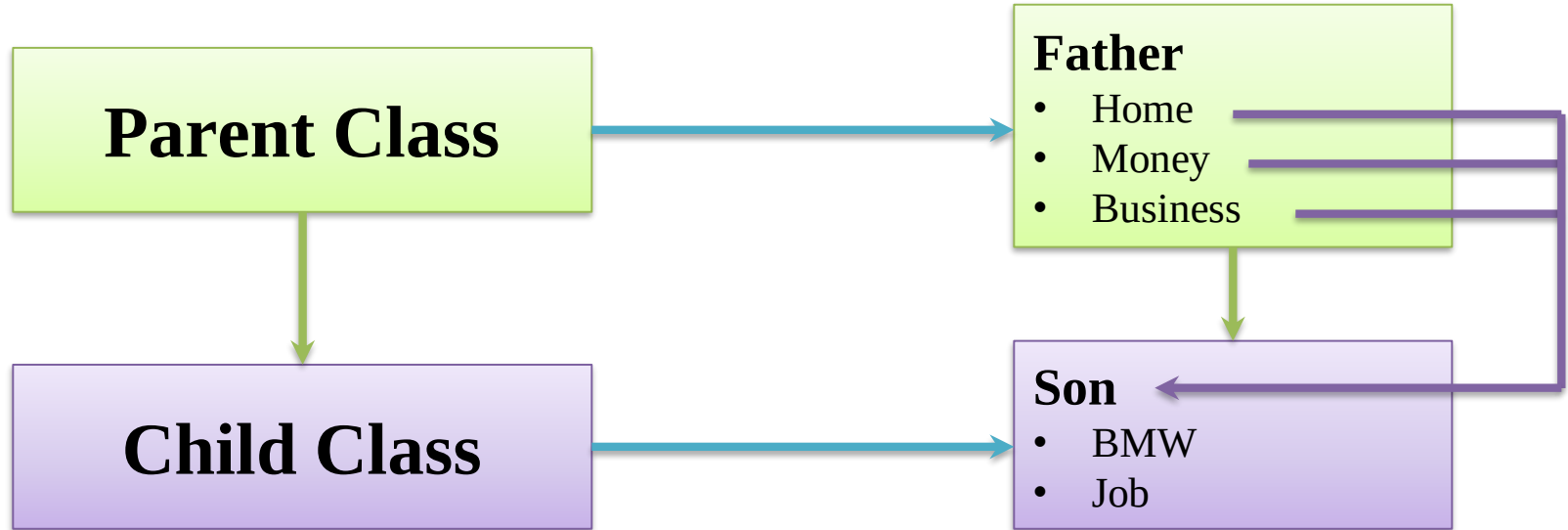
The mechanism of deriving a new class from an old one (existing class) such that the new class inherit all the members (variables and methods) of old class is called inheritance or derivation.



Super Class and Sub Class

The old class is referred to as the Super class and the new one is called the Sub class.

- Parent Class - Base Class or Super Class
- Child Class - Derived Class or Sub Class



Parent Class

```
class Employee :  
    id = 1  
    @classmethod  
    def getid(cls):  
        return cls.id  
    def setname(self, name):  
        self.name = name  
    def getname(self):  
        return self.name  
    def setsalary(self, salary):  
        self.salary = salary  
    def getsalary(self):  
        return self.salary  
    def setovertime(self, ot):  
        self.ot = ot  
    def getovertime(self):  
        return self.ot
```

Child Class

```
class Manager :  
    def setsalary(self, salary):  
        self.salary = salary  
    def getsalary(self):  
        return self.salary  
    def getseniorname(self, sname):  
        self.sname = sname  
    def getseniorname(self):  
        return self.sname
```

Type of Inheritance

- Single Inheritance
- Multi-level Inheritance
- Hierarchical Inheritance
- Multiple Inheritance

Declaration of Child Class

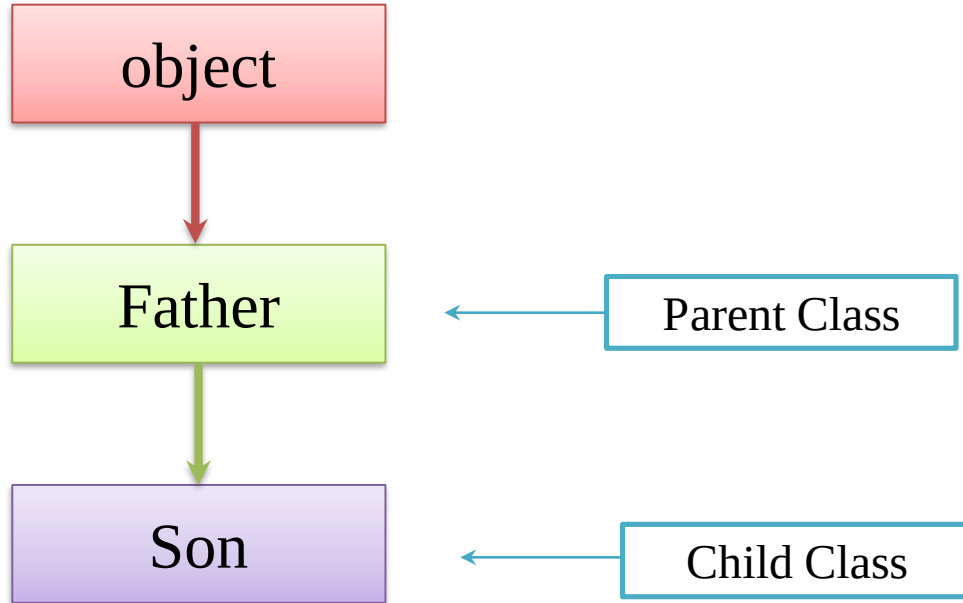
class ChildClassName (ParentClassName) :
 members of Child class

class Mobile (object) :
 members of Child class

class Mobile :
 members of Child class

Single Inheritance

If a class is derived from one base class (Parent Class), it is called Single Inheritance.



Syntax:-

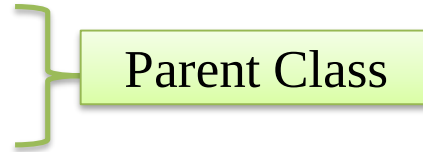
```
class ParentClassName(object):  
    members of Parent Class
```

```
class ChildClassName(ParentClassName):  
    members of Child Class
```

Example:-

```
class Father:  
    members of class Father
```

```
class Son (Father):  
    members of class Son
```



Inheritance

- We can access Parent Class Variables and Methods using Child Class Object
- We can also access Parent Class Variables and Methods using Parent Class Object
- We can not access Child Class Variables and Methods using Parent Class Object

Constructor in Inheritance

By default, The constructor in the parent class is available to the child class.

class Father:

```
def __init__(self):  
    self.money = 2000  
    print("Father Class Constructor")
```

What will happen if we define
constructor in both classes ?

class Son (Father):

```
def disp(self):  
    print("Son Class Instance Method:",self.money)
```

s = Son()

s.disp()

s.money

Constructor Overriding

If we write constructor in the both classes, parent class and child class then the parent class constructor is not available to the child class.

In this case only child class constructor is accessible which means child class constructor is replacing parent class constructor.

Constructor overriding is used when programmer want to modify the existing behavior of a constructor.

Constructor Overriding

```
class Father:
```

```
    def __init__(self):  
        self.money = 2000  
        print("Father Class Constructor")
```

```
class Son(Father):
```

```
    def __init__(self):  
        self.money = 5000  
        print("Son Class Constructor")  
    def disp(self):  
        print(self.money)
```

```
s = Son()
```

```
s.disp()
```

How can we call parent
class constructor ?

Constructor with super() Method

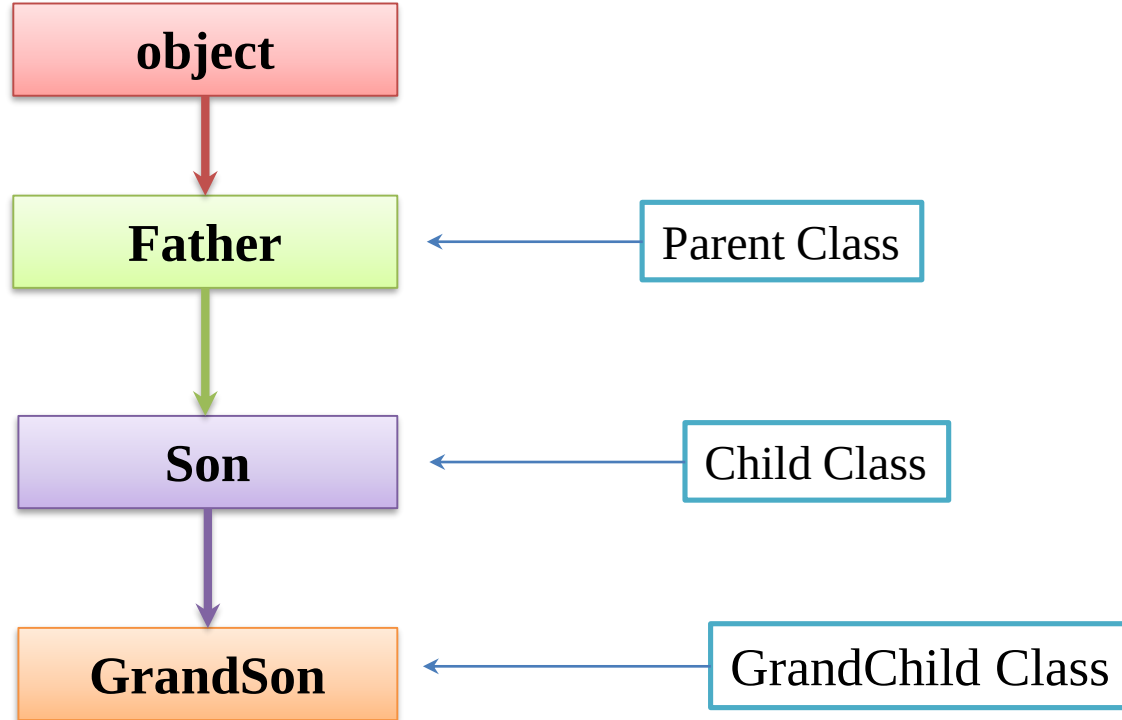
If we write constructor in the both classes, parent class and child class then the parent class constructor is not available to the child class.

In this case only child class constructor is accessible which means child class constructor is replacing parent class constructor.

super () method is used to call parent class constructor or methods from the child class.

Multi-level Inheritance

In multi-level inheritance, the class inherits the feature of another derived class (Child Class).



class Father (object):
members of class Father

class Son (Father):
members of class Son

class GrandSon (Son):
members of class GrandSon

} Parent Class

} Child Class

} GrandChild Class

object

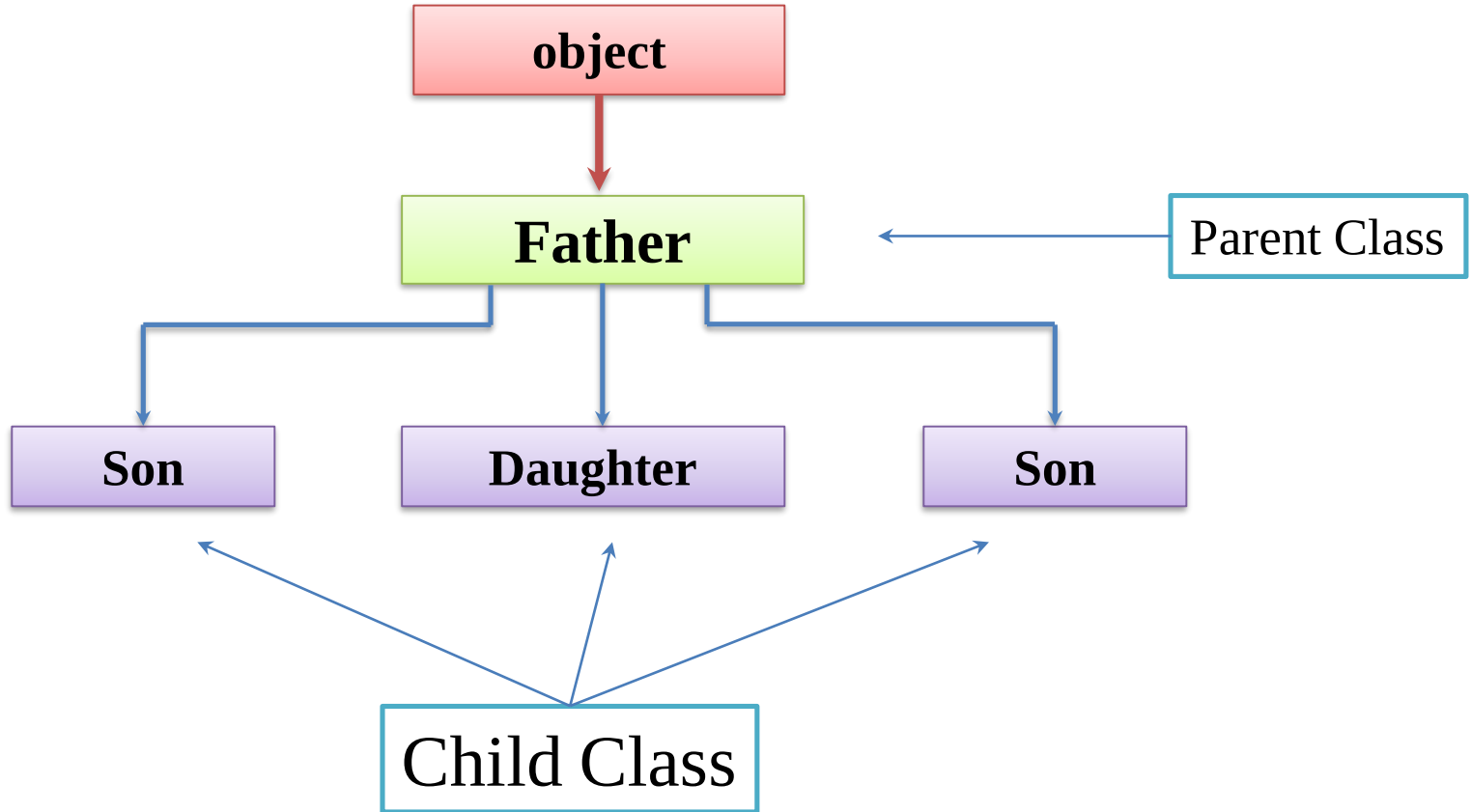
Father

Son

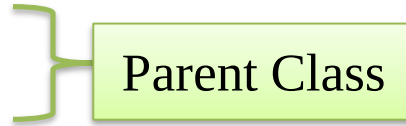
GrandSon



Hierarchical Inheritance



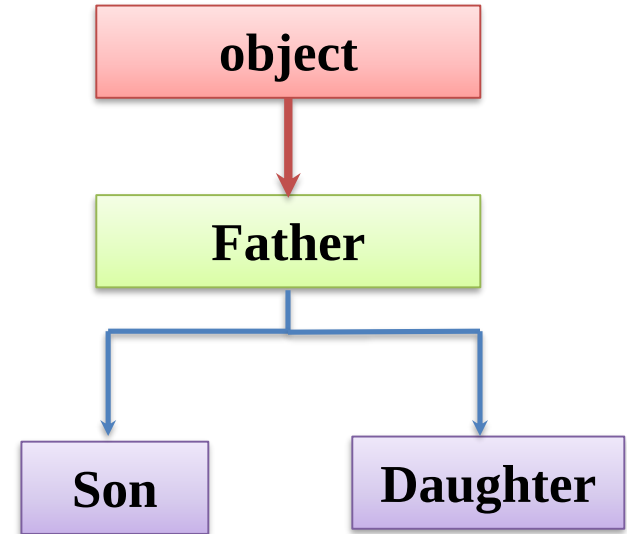
class Father (object):
members of class Father



class Son (Father):
members of class Son

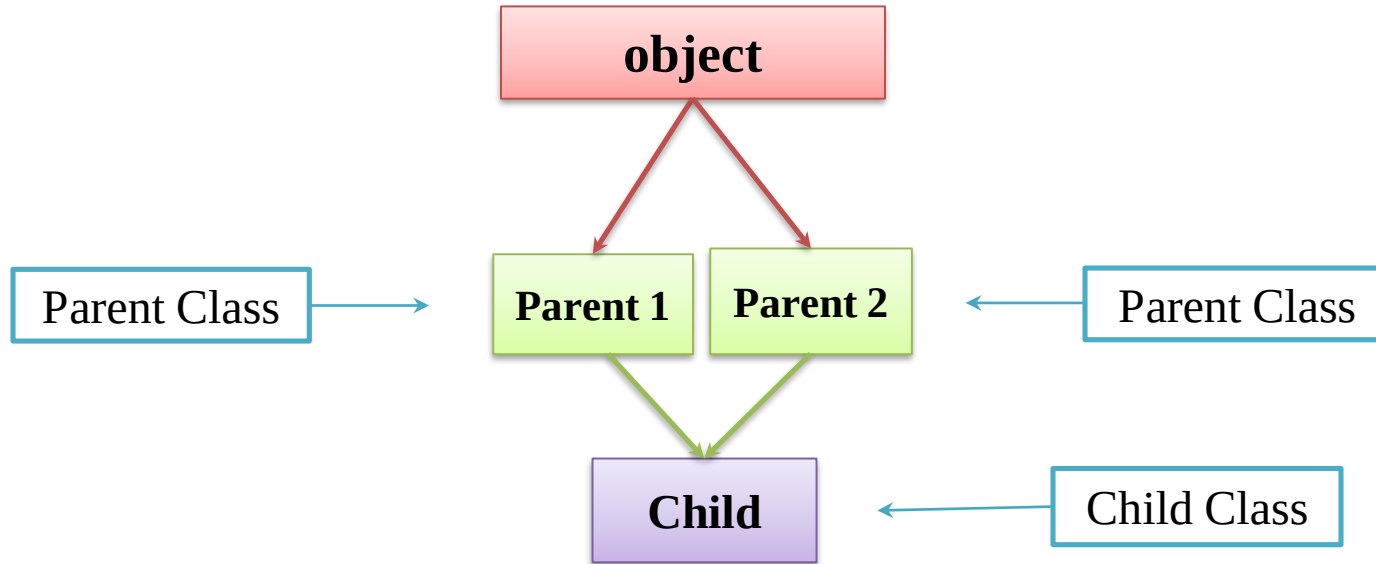


class Daughter (Father):
members of class Daughter

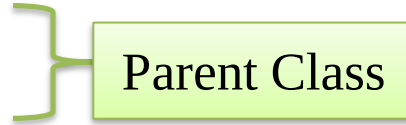


Multiple Inheritance

If a class is derived from more than one parent class, then it is called multiple inheritance.



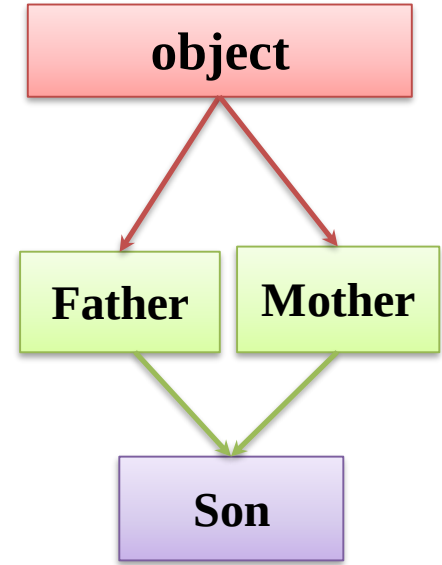
class Father (object):
members of class Father



class Mother (object):
members of class Mother



class Son (Father, Mother):
members of class Son



Method Resolution Order (MRO)

`s = Son()`

- The search will start from Son. As the object of Son is created, the constructor of Son is called.
- Son has `super().__init__()` inside his constructor so its parent class, the one in the left side 'Father' class's constructor is called.
- Father class also has `super().__init__()` inside his constructor so its parent 'object' class's constructor is called.
- Object does not have any constructor so the search will continue down to right hand side class (Mother) of object class so Mother class's constructor is called.
- As Mother class also has `super().__inti__()` so its parent class 'object' constructor is called but as object class already visited, the search will stop here.

